Spam Email Classifier

From email data to a working classifier

A personal project report

Project details	
Author	Mohamed Aassou
Project	Spam Email Classifier
Source	GitHub repository

Abstract

This report describes my personal Spam Email Classifier project. I prepared a public collection of emails, compared several ways to distinguish spam from legitimate messages, and built a small web page that shows a prediction. The examined dataset contains 2,998 messages before repeated emails are removed. On the reserved final test set, the chosen model classified 586 of 590 messages correctly. The report explains what these results show, where the current application stops, and why performance on this older collection may not reflect present-day email.

Contents

Abstract i

1 Introduction and project scope 1

1.1 Why this project matters . . . 1

1.2 Project goals . . . 1

1.3 Technologies used . . . 2

2 The email data 3

2.1 Source and balance . . . 3

2.2 Preparing the messages . . . 3

3 How the models were compared 5

3.1 Keeping evaluation separate . . . 5

3.2 Choosing the final model . . . 5

4 What the evaluation showed 7

4.1 Comparing the models . . . 7

4.2 Testing the chosen model . . . 7

5 Using the classifier 12

5.1 From email to result . . . 12

5.2 Current limits of the application . . . 12

6 Limits and next steps 14

7 Conclusion 15

A Supplementary visual material 16

References 17

List of Figures

2.1 Messages in the examined dataset before repeated emails are removed. Legitimate mail makes up 83.4% of the total. 4

3.1 Learning process: compare models, train the chosen one again, then run the final test. 6

4.1 Comparison results for the models before the final test. Logistic Regression and SVM have the same scores in this figure. 8

4.2 Validation results for the majority-choice starting point: it misses all 76 spam emails. 8

4.3 Naive Bayes comparison: eight spam emails caught and 68 missed. 9

4.4 Logistic Regression comparison: one false alarm and six missed spam emails. 9

4.5 Validation results for SVM: the same recorded mistake counts as Logistic Regression. 10

4.6 Final test results: 493 legitimate and 93 spam emails classified correctly, with four mistakes in total. 10

4.7 Final test curve showing how well the chosen model separates spam from legitimate mail at different decision levels. 11

5.1 The email prediction journey. Showing a spam probability is still planned. . . 13

5.2 Home page with spaces for the email subject and body and a panel for the result. 13

A.1 Another view of the prediction journey. The probability display is planned. . 16

Chapter 1

Introduction and project scope

1.1 Why this project matters

Spam wastes attention and can contain deceptive offers or requests. A filter can help, but its mistakes matter: a legitimate message might be marked as spam, while an unwanted one might still reach the inbox. I built this project to learn how an email classifier is prepared, evaluated and made available through a simple web page. A person enters a subject and message body and receives a spam or legitimate result. The project does not connect to an inbox or move messages automatically.

The work uses two parts of the public SpamAssassin email collection: one containing ordinary messages and one containing spam[1]. An early project note names 97% accuracy as a goal. The results in this report come from the saved evaluation figures and are presented separately from that goal. The source is available in the project's GitHub repository.

1.2 Project goals

The main goal is to make a useful prediction from an email's subject and body. I also wanted a fair comparison between a simple starting point and several learning models. Table 1.1 shows what the current project does and what is still planned.

Table 1.1: Project goals and current state

Goal	What it means	State
Enter an email	Supply a subject and body; reject empty entries.	Available
Classify it	Return spam or legitimate mail.	Available
See the answer	Show the result on the web page.	Available
Show confidence	Display the estimated chance of spam.	Planned
Compare fairly	Choose a model before using the reserved final test set.	Available
Measure speed	Record how long predictions take.	Planned

1.3 Technologies used

The project uses a small set of tools for preparing email data, comparing models, showing results and serving the web page. Their official logos are shown in Table 1.2.

Table 1.2: Main technologies used in the project

Official logo	Role in the project
	**Python** is the language used to build the classifier and the web application.
	**pandas** helps read, inspect and prepare the email collection.
	**scikit-learn** provides the text features, learning models and evaluation tools.
	**FastAPI** connects the web page to the saved classifier.
	**Matplotlib** helps create charts for data exploration and model evaluation.

The project also uses NLTK to prepare words, Jinja2 to display the web page and Uvicorn to run the web service.

Chapter 2

The email data

2.1 Source and balance

The project draws on two publicly available SpamAssassin collections[1]. It extracts each email's subject and plain-text body and stores them for analysis. The dataset examined for this report contains 2,998 emails: 2,500 legitimate messages and 498 spam messages. Figure 2.1 shows this imbalance. A classifier that always says "legitimate" would be right often, but it would fail to catch every spam email. That is why the later comparison looks beyond overall accuracy.

The examined data also contains 49 repeated subject-and-body pairs, two empty subjects and 212 empty bodies. No identical pair carries conflicting labels. These numbers describe the saved dataset used for inspection; a fresh download or different parsing method could produce different counts.

2.2 Preparing the messages

Missing subjects or bodies are treated as empty text. The project makes labels consistent, checks for contradictory labels on identical emails, and removes repeated subject-and-body pairs. This leaves 2,949 distinct pairs: 2,472 legitimate messages and 477 spam messages. Removing exact repeats before dividing the data prevents the same email from appearing in both learning and evaluation groups.

For each remaining email, the subject and body are joined. The text is changed to lowercase, punctuation and numbers are removed, common English words are dropped, and related word forms are reduced to a common stem. A separate word-weighting step then gives more influence to words that help distinguish messages; this method is known as TF-IDF[3]. The word weights are learned only from the messages used to train a model[2]. Similar but non-identical emails can still appear in different groups.

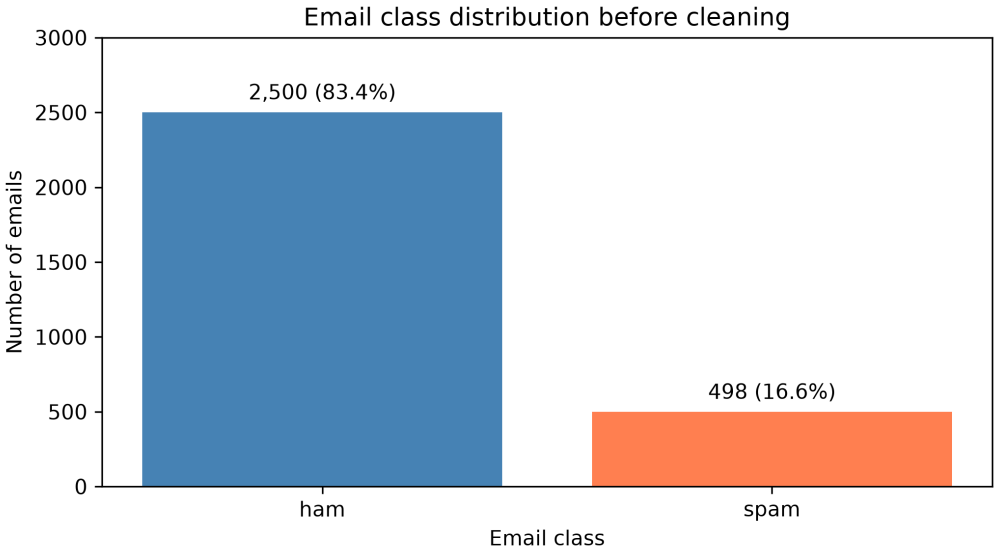


Figure 2.1: Messages in the examined dataset before repeated emails are removed. Legitimate mail makes up 83.4% of the total.

Chapter 3

How the models were compared

3.1 Keeping evaluation separate

The prepared emails are divided into three groups while keeping a similar share of spam in each. About 64% are used to fit the candidate models, 16% to compare them, and 20% are held back for a final test. The comparison group contains 472 emails (396 legitimate, 76 spam); the final test group contains 590 (495 legitimate, 95 spam). Figure 3.1 gives a visual overview. It simplifies the process, so the three group sizes and the final retraining step are explained here.

The starting point is a model that always chooses the most common group. Three other approaches use weighted words: Naive Bayes, Logistic Regression and a linear Support Vector Machine (SVM). Each one learns from the first group, then makes predictions for the separate comparison group. The word weights are learned alongside each model, so the comparison and final test messages do not influence that learning[2].

3.2 Choosing the final model

The comparison uses three measures focused on spam. *Precision* asks how often a spam warning is right. *Recall* asks how much real spam is caught. *F1* combines both into one score. The project uses Logistic Regression as its final model. This choice is set in the project rather than selected automatically from the comparison results. The model is then trained again using the combined 80% learning portion and saved for later predictions. Only after that does the project evaluate it on the untouched final test group.

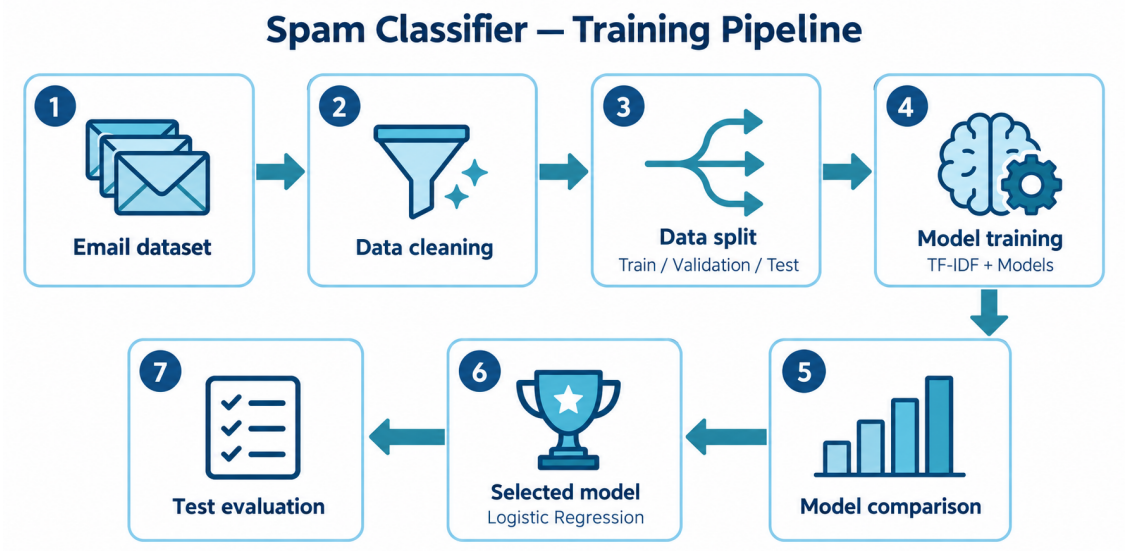


Figure 3.1: Learning process: compare models, train the chosen one again, then run the final test.

Chapter 4

What the evaluation showed

4.1 Comparing the models

Figure 4.1 compares the models on the same validation group. The simple model that always chooses the majority class catches no spam. Naive Bayes catches eight of 76 spam emails but misses the other 68. Logistic Regression and SVM perform equally in the saved results: both catch 70 spam emails, miss six, and wrongly flag one legitimate email. Their displayed precision, recall and F1 scores are therefore tied. The project proceeds with Logistic Regression, which can also estimate a chance of spam, although the current web page does not show that estimate.

Figures 4.2–4.5 show the mistakes behind the summary chart. In each matrix, the rows are the real labels and the columns are the predicted labels. These figures belong to the model comparison stage; they are separate from the final test.

4.2 Testing the chosen model

On the reserved test group, the chosen model classifies 493 legitimate emails and 93 spam emails correctly. It wrongly flags two legitimate emails and misses two spam emails, as Figure 4.6 shows. That is 586 correct answers out of 590, or about 99.32% accuracy. For spam specifically, both precision and recall are about 97.89%; F1 is also about 97.89%. These results apply to this test group and do not establish how well the model would handle today's incoming mail.

Figure 4.7 shows how the model separates the two groups across different decision levels. Its displayed summary score rounds to 1.00. The figure does not provide the more precise value, so none is claimed here.

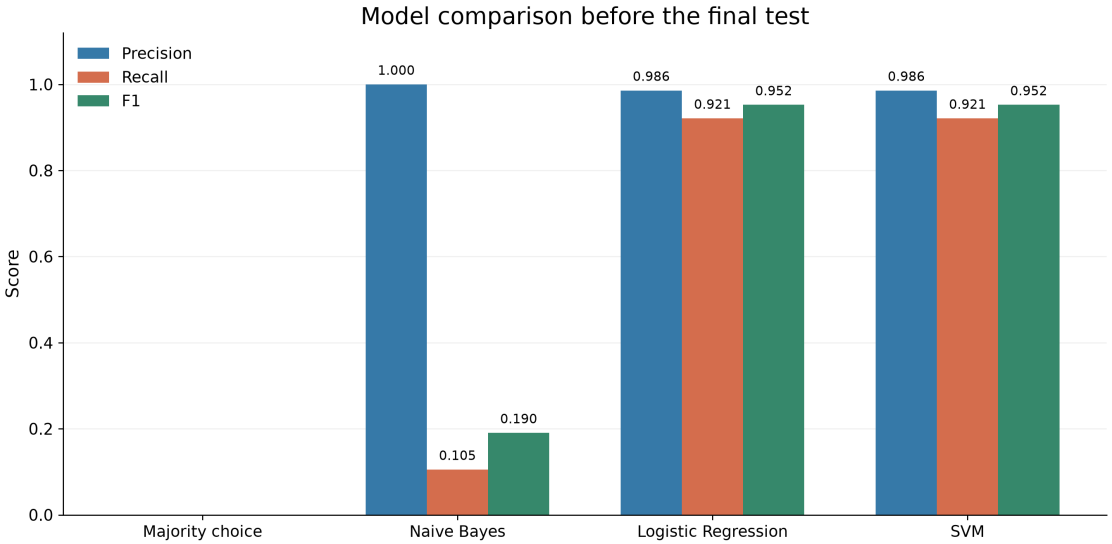


Figure 4.1: Comparison results for the models before the final test. Logistic Regression and SVM have the same scores in this figure.

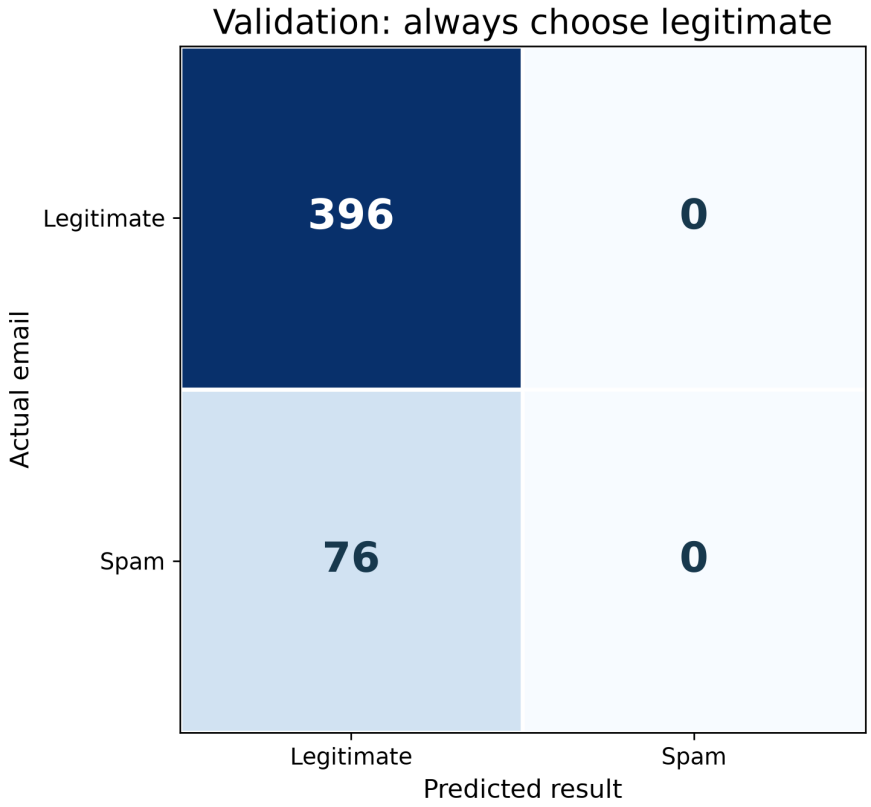


Figure 4.2: Validation results for the majority-choice starting point: it misses all 76 spam emails.

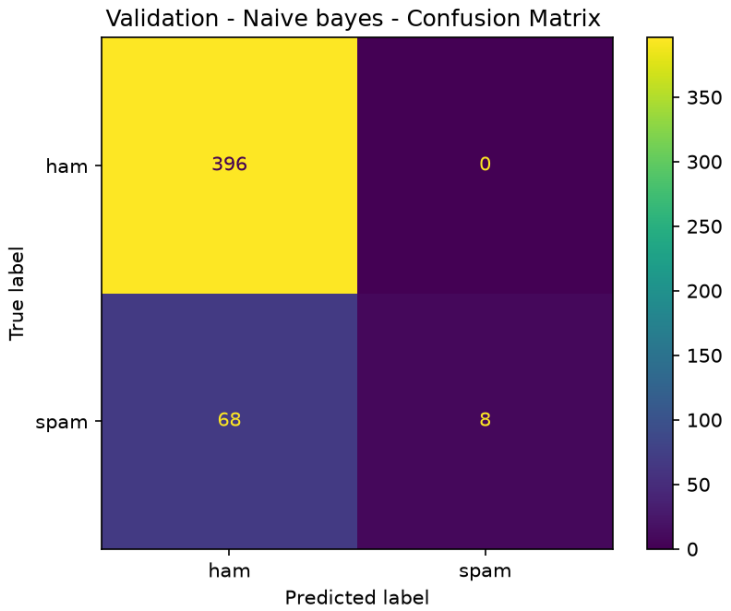


Figure 4.3: Naive Bayes comparison: eight spam emails caught and 68 missed.

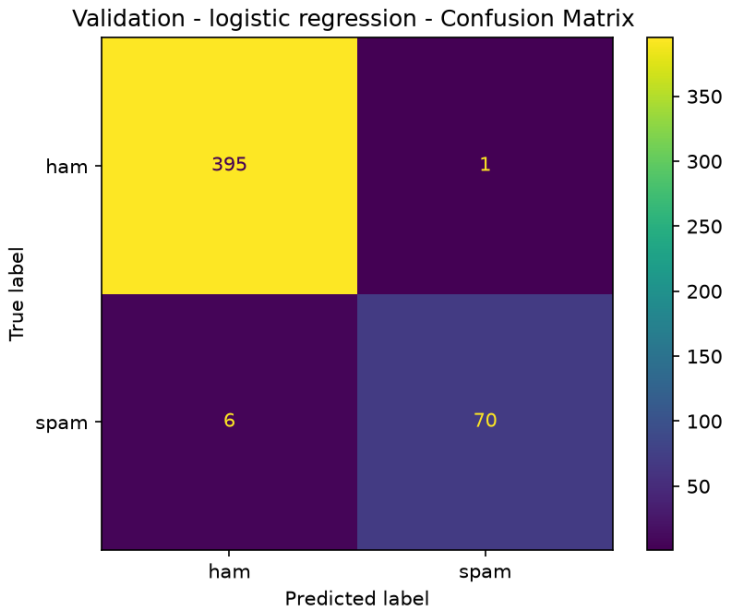


Figure 4.4: Logistic Regression comparison: one false alarm and six missed spam emails.

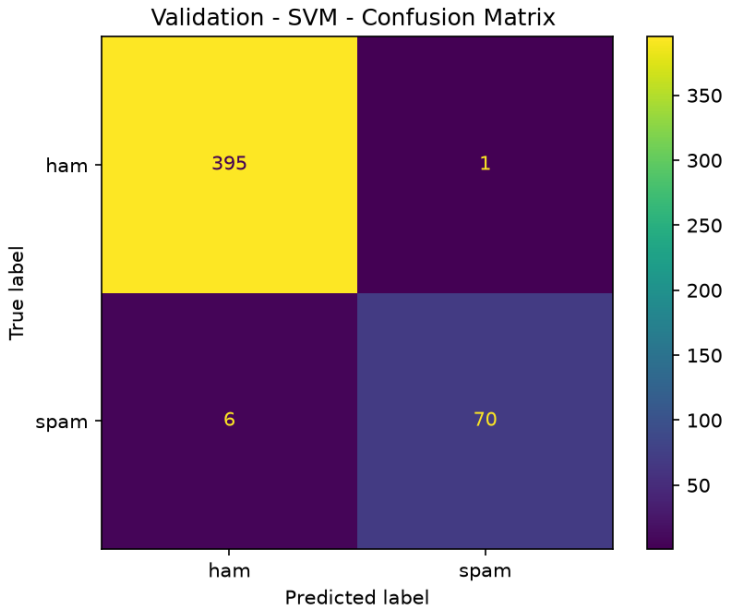


Figure 4.5: Validation results for SVM: the same recorded mistake counts as Logistic Regression.

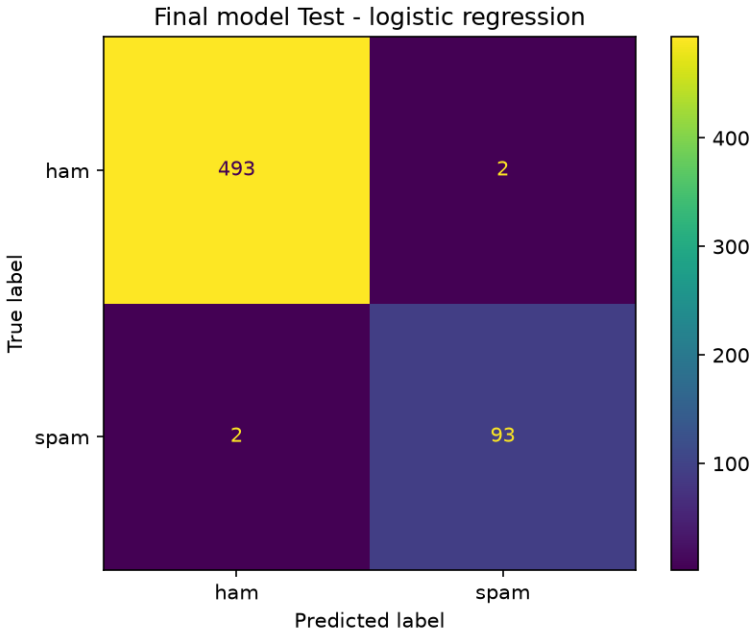


Figure 4.6: Final test results: 493 legitimate and 93 spam emails classified correctly, with four mistakes in total.

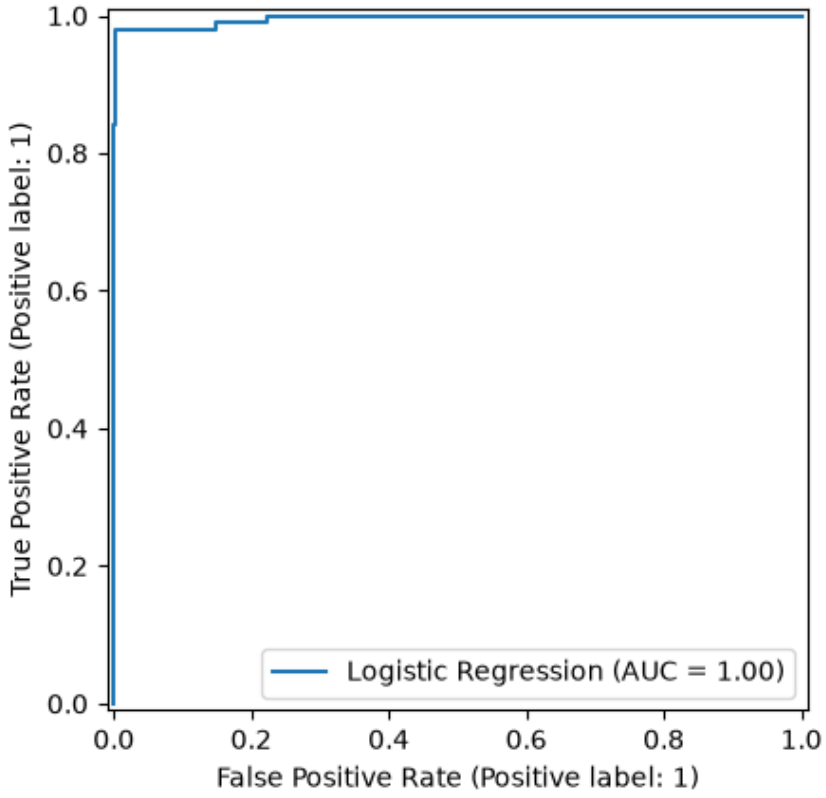


Figure 4.7: Final test curve showing how well the chosen model separates spam from legitimate mail at different decision levels.

Chapter 5

Using the classifier

5.1 From email to result

The web page asks for a subject and message body. It checks that neither field is empty, then sends the text to a small FastAPI service. The service prepares the text in the same general way as the training data, asks the saved model for a prediction, and returns “spam” or “legitimate.” The page then shows the result with a short explanation. A chance of spam can be calculated inside the application, but it is not currently included in what the user sees.

Figure 5.1 illustrates that path. Its final “Spam probability” label describes a planned addition, not the current screen. Figure 5.2 shows the page before an email has been checked. It documents the interface rather than a completed prediction.

5.2 Current limits of the application

The application reads the saved model again for each prediction. That makes the design simple, but it may add delay; no timing study has been recorded. The project has a local web interface, with no evidence here of a hosted service. A few example emails are included for demonstration, but they are not a substitute for the separate evaluation set.

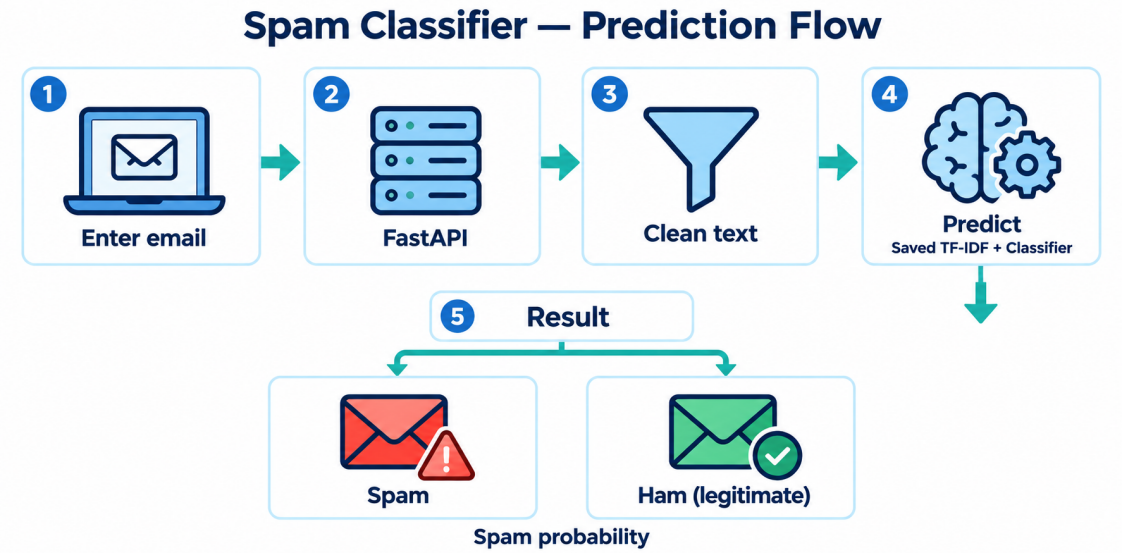


Figure 5.1: The email prediction journey. Showing a spam probability is still planned.

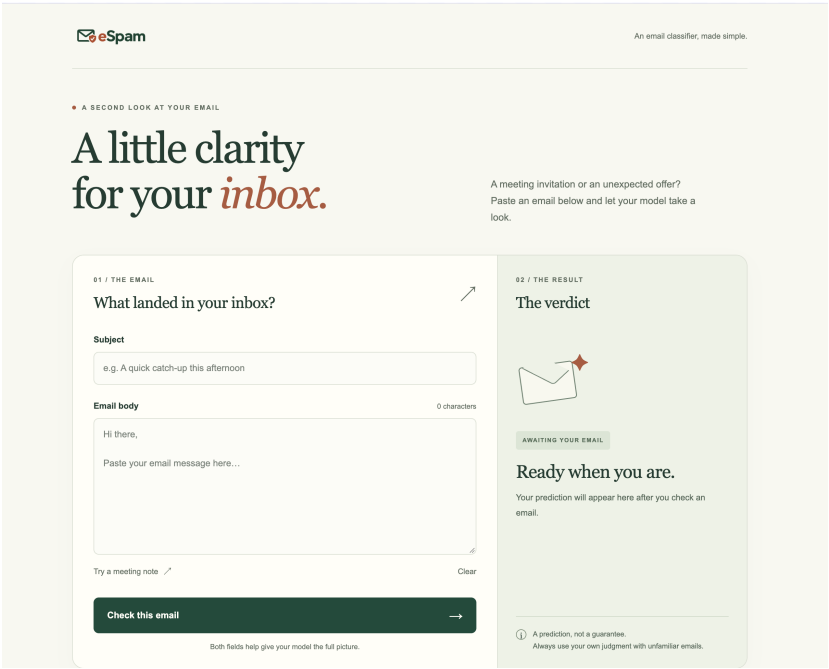


Figure 5.2: Home page with spaces for the email subject and body and a panel for the result.

Chapter 6

Limits and next steps

The final test contains two legitimate emails marked as spam and two spam emails allowed through. The saved chart shows how many mistakes occurred, but the project does not yet examine the actual messages behind them. Reviewing those emails would show whether particular words, writing styles or message lengths caused trouble. It would also help decide whether the classifier should be more cautious about blocking legitimate mail or more aggressive about catching spam.

The data comes from an older public collection. Its legitimate messages come from the "easy ham" group, which the corpus notes is relatively easy to distinguish from spam[1]. The strong test result may therefore be optimistic for modern or unfamiliar email. The project has no separate collection from another time or source for a broader check. Repeated identical emails were removed before evaluation, but very similar messages may remain in different groups.

There is also a small difference between learning and prediction: the prepared learning messages include words identifying the subject and body, while the text sent for a new prediction does not. The project should prepare both in exactly the same way. Future work could also show the estimated chance of spam, examine different decision levels, study mistakes one message at a time, record prediction speed, and test on newer email. These are next steps, not features claimed as complete.

Chapter 7

Conclusion

This personal project follows the full path from collecting email data to showing a classification on a web page. The saved comparison shows that the learned models improve greatly on always choosing the most common group. Logistic Regression and SVM tie in the displayed comparison, and the chosen Logistic Regression model makes four mistakes among 590 messages in the reserved final test. The result is promising for this collection, while the age and limited variety of the data leave an important question: how well would it work on new kinds of email? A broader test and a closer look at mistakes are the most useful next steps.

Appendix A

Supplementary visual material

Figure A.1 shows another version of the prediction journey. Its "Spam probability" label refers to a planned feature. The current page shows only the classification.

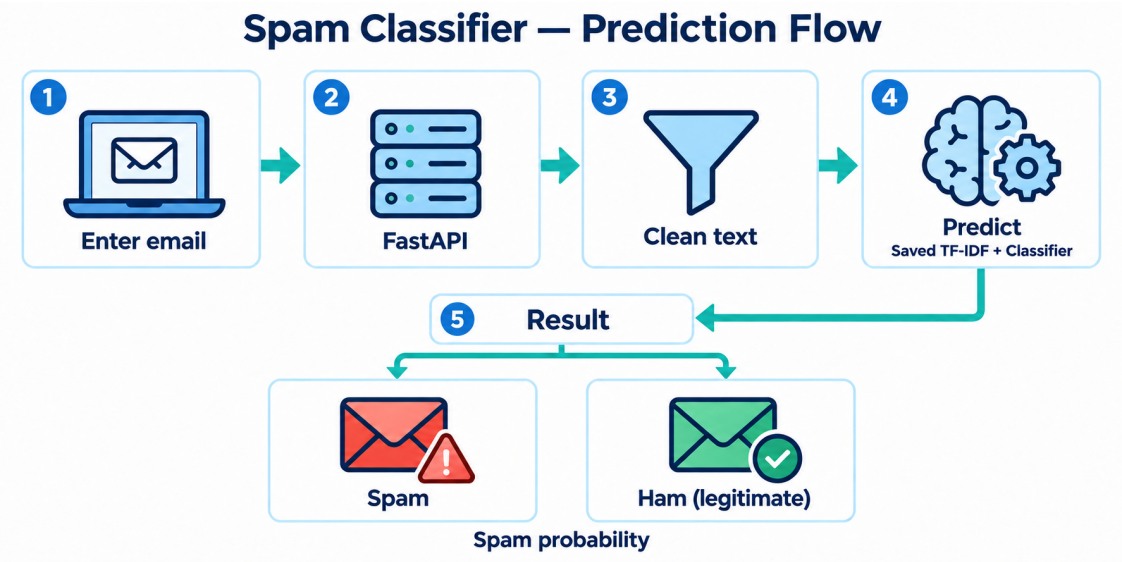


Figure A.1: Another view of the prediction journey. The probability display is planned.

Bibliography

[1] Apache SpamAssassin, "Public Corpus README" and archive index. Accessed 28 October 2026.

[2] scikit-learn developers, "Getting Started: Estimators and pipelines." Accessed 28 October 2026.

[3] scikit-learn developers, "TfidfVectorizer documentation." Accessed 28 October 2026.